



Motor de Tarifas Orientado a Objetos

RETO PRÁCTICO • VERSIÓN 3

¡Bienvenidos a la recta final de la estructuración de datos! Hasta ahora, nuestro motor de tarifas funcionaba mediante arreglos y bucles sueltos. Sin embargo, para construir aplicaciones escalables y de nivel empresarial, necesitamos elevar nuestra arquitectura.

A partir del estándar ES6, JavaScript introdujo las **clases** como una forma mucho más estructurada y declarativa para gestionar la creación de objetos y sus comportamientos. Tu nueva misión es **refactorizar todo el sistema** utilizando Clases, encapsulamiento y desestructuración.

Objetivos a Evaluar

Este reto cubre cuatro competencias fundamentales de la programación orientada a objetos en JavaScript moderno. Cada objetivo será evaluado de forma independiente y debe estar correctamente implementado en tu entrega final.

1

Clases y Constructores (ES6)

Creación de moldes para instanciar objetos utilizando la palabra clave `class`, el método `constructor` y la instanciación con `new`.

2

Encapsulamiento y Privacidad

Protección de datos sensibles utilizando propiedades privadas con el prefijo `#`, garantizando que el estado interno crítico jamás sea alterado desde el exterior.

3

Métodos y Contexto (`this`)

Definición de comportamientos internos que interactúan directamente con las propiedades de la instancia mediante la palabra clave `this`.

4

Desestructuración

Extracción elegante de propiedades para generar la Boleta Virtual con un código más limpio y sin repetición de notación de punto.

Fase 1: La Clase Pasajero

MODELANDO LA ENTIDAD

Deberán crear una clase llamada `Pasajero` que servirá como molde para cada cliente del sistema. Esta clase concentra toda la lógica relacionada con un pasajero individual.

El Constructor

Debe recibir y asignar las propiedades públicas: `nombre`, `edad`, `destino` y `tieneTNE`. Estas propiedades son accesibles desde el exterior de la clase.

Privacidad Estricta

El precio base del pasaje (ej. \$5000) debe ser declarado como una **propiedad privada** (`#precioBase`) al inicio de la clase. Esto garantiza que el estado interno crítico jamás sea alterado desde el exterior.

Método `calcularTarifaFinal()`

Método público que usa `this` para evaluar la edad y el estado del TNE del pasajero, aplicando los descuentos correspondientes sobre el `#precioBase` y retornando el valor final a pagar.



Fase 2: La Clase Sistema De Ventas

GESTIÓN CENTRALIZADA

En lugar de tener bucles sueltos en el archivo, encapsularemos la lógica de negocio en una segunda clase llamada `SistemaDeVentas`. Al almacenar los métodos en la clase, se guardan centralizadamente en el prototipo, **optimizando la memoria** de la aplicación.

Estado Interno

El constructor no recibe parámetros, pero debe inicializar una propiedad `this.listaPasajeros` como un arreglo vacío que almacenará todas las instancias registradas.

Métodos Principales

- 1 registrarPasajero(pasajero)**
Recibe una instancia de `Pasajero`. Si el destino **NO es "Rancagua"**, rechaza el registro e imprime un error en consola. Si es válido, lo añade al arreglo.
- 2 procesarBoletas()**
Itera sobre `this.listaPasajeros` para generar los cobros y boletas virtuales de cada pasajero registrado en el sistema.

Fase 3 y 4: Desestructuración e Instanciación

Las últimas dos fases conectan la lógica interna con la ejecución real del sistema. Primero se define cómo se genera la boleta, y luego se simula el entorno de producción completo.

Fase 3 · Boleta Virtual

Dentro de `procesarBoletas()`, queda **estrictamente prohibido** usar la notación de punto repetitivamente (ej. `pasajero.nombre`). Deben utilizar la **desestructuración**:

```
const { nombre, destino } = pasajero;
```

Luego, impriman la Boleta Virtual en consola interpolando las variables desestructuradas y llamando al método `calcularTarifaFinal()` mediante **Template Literals**.

Fase 4 · Ejecución

Fuera de las clases, simulen el entorno de producción siguiendo estos pasos:

01

Instanciar el sistema

```
const terminalSantiago = new  
SistemaDeVentas();
```

02

Crear pasajeros

Instancien al menos **4 pasajeros diferentes** utilizando la palabra clave `new`.

03

Registrar y procesar

Registren a los pasajeros en el sistema y ejecuten el procesamiento de boletas.

Lógica de Negocio: Descuentos y Validaciones

El motor de tarifas debe aplicar reglas de negocio precisas a través de los métodos de las clases. Estas reglas son el corazón funcional del sistema y deben operar de manera impecable.

1

Tercera Edad

50% de descuento

aplicado sobre el `#precioBase` para pasajeros que califiquen por edad.

Evaluado dentro de `calcularTarifaFinal()` usando `this.edad`.

2

Tarjeta TNE

20% de descuento

aplicado sobre el `#precioBase` para pasajeros con `tieneTNE = true`.

Evaluado dentro de `calcularTarifaFinal()` usando `this.tieneTNE`.

3

Validación de Destino

Solo se aceptan pasajeros con destino "**Rancagua**".

El método `registrarPasajero()` debe rechazar cualquier otro destino e imprimir el error correspondiente en consola.

⚠ Los descuentos y la validación de destino deben funcionar exclusivamente a través de los métodos de las clases. No se permiten variables ni funciones globales sueltas fuera de ellas.

Entregables y Criterios de Calidad

Deberán subir su archivo `.js` debidamente comentado, explicando sus decisiones técnicas. La evaluación se realizará sobre cuatro criterios fundamentales que reflejan el dominio de la programación orientada a objetos en ES6.

Criterio	Descripción
Arquitectura POO	Correcta implementación de <code>class</code> , <code>constructor</code> y la instanciación con <code>new</code> . El código ya no debe tener variables ni funciones globales sueltas.
Encapsulamiento	Uso obligatorio y funcional del símbolo <code>#</code> para ocultar la tarifa base, demostrando control sobre la privacidad de los datos.
Desestructuración	Extracción limpia de datos en la generación de boletas, reduciendo el ruido visual del código sin usar notación de punto repetitiva.
Lógica de Negocio	Los descuentos (Tercera edad 50%, TNE 20%) y la validación de destino ("Rancagua") deben funcionar de manera impecable a través de los métodos de las clases.